INF201 - Lecture 6

12th Oct 2026

Topics

- Deliverables
- A reminder of GitHub collaborative work flow
- Regular expressions
- Representing data in Python

In [ ]:

Deliverables

- The first deliverable must be submitted by Friday at 16
- As the exercise classes are no longer mandatory, they are less full, so feel free attend classes other
  than the one you are signed up for if you want more support, or maybe just a place to work
- I have created a new set of groups on Canvas under the heading Deliverables
  - These are called something like this: G1_4
  - The deliverables are group assignments, meaning that one of you submit on behalf of both/all of you
  - Communicate with each other to make sure the assignment is submitted on time!
  - Check that you have uploaded the files correctly!
- Remember that we asked you to make a new virtual environment (venv) for each of the deliverables
  - To show that you have done so, include a requirements.txt (created by `pip freeze`) in your repo
- Please use English for these assignments!

A quick reminder of how to collaborate in GitHub

- Create the repository *before* you start working on the code: This way everything gets tracked
- Use branches rather than working directly on main
- This will give fewer conflicts than if you are both editing the same document on main
- Merge the branch back to main when it is working well

- Remember to commit, pull and push *often*: This keeps your local repo up to date with the remote

Regular expressions

An example for motivation

- You are given a text file and asked to extract all email addresses mentioned in the file.
- How would you do this in Python?

In [3]:
```
text = open("unstructured_leads.txt").read()  # File generated by Gemini
print(text)
```

```
Ticket #101: Please email John at john.doe@techcorp.com or call 555-123-4567.
Ticket #102: Client SARAH (s.connor@cyberdyne.io.) reported an issue with login.
Ticket #103: Urgent! Reach out to alex_99@dev-labs.org or call +1-555-999-8888!
Ticket #104: System backup failed. No user contact listed.
Ticket #105: Contact options: [admin@hospital.net] or direct line 555.333.4444.
```

1. Read file (line by line?)
2. Split the string into "words"

In [4]:
```
# words = text.split()
# print(words)

with open("unstructured_leads.txt") as f:
    for line in f:
        words = line.split()
        print(words)
```

```
['Ticket', '#101:', 'Please', 'email', 'John', 'at', 'john.doe@techcorp.com', 'o
r', 'call', '555-123-4567.']
['Ticket', '#102:', 'Client', 'SARAH', '(s.connor@cyberdyne.io.)', 'reported', 'a
n', 'issue', 'with', 'login.']
['Ticket', '#103:', 'Urgent!', 'Reach', 'out', 'to', 'alex_99@dev-labs.org', 'o
r', 'call', '+1-555-999-8888!']
['Ticket', '#104:', 'System', 'backup', 'failed.', 'No', 'user', 'contact', 'list
ed.']
['Ticket', '#105:', 'Contact', 'options:', '[admin@hospital.net]', 'or', 'direc
t', 'line', '555.333.4444.']
```

3. Split gave us the "words" and removed the spaced between them, but some punctuation still remains and needs to be removed: `[`, `]`, `(`, `)`, `.`, `:`, `!`
4. We are looking for email addresses, so we only keep to keep "words" with `@` in them

In [5]:
```
words = text.split()
for word in words:
    if "@" in word:
        cleaned = word.strip(".:!()[]")
        print(cleaned)
```

```
# with open("unstructured_leads.txt") as f:
#     for line in f:
#         words = line.split()
#         for word in words:
#             if "@" in word:
#                 cleaned = word.strip(".()[]")
#                 print(cleaned)
```

```
john.doe@techcorp.com
s.connor@cyberdyne.io
alex_99@dev-labs.org
admin@hospital.net
```

5. This looks nice! However, we should also verify that these "words" are formatted like an email address, in case we want to use the script on a different file

In [6]:
```
emails_found = []
words = text.split()
for word in words:
    if "@" in word:
        cleaned = word.strip(".:!()[]")
        if cleaned.count("@") == 1:
            parts = cleaned.split("@")
            if parts[0] and "." in parts[1]:
                # Check that the first part is non-empty and that the second par
                emails_found.append(cleaned)
emails_found

# emails_found = []
# with open("unstructured_leads.txt") as f:
#     for line in f:
#         words = line.split()
#         for word in words:
#             if "@" in word:
#                 cleaned = word.strip(".()[]")
#                 if cleaned.count("@") == 1:
#                     parts = cleaned.split("@")
#                     if parts[0] and "." in parts[1]:
#                         emails_found.append(cleaned)
# emails_found
```

Out[6]:
```
['john.doe@techcorp.com',
 's.connor@cyberdyne.io',
 'alex_99@dev-labs.org',
 'admin@hospital.net']
```

- Nice! We solved the problem with 9 lines of code (10 if we include reading the file).
- Now, could we easily modify the code to look for phone numbers instead?

In [7]:
```
phones_found = []
words = text.split()
for word in words:
    # Clean off trailing sentence punctuation
    cleaned = word.strip(".:!()[]")

    # Count actual digits
    digit_count = sum(c.isdigit() for c in cleaned)
```

```
    # Check if all characters are valid phone symbols (+, -, ., digits)
    valid_chars = all(c.isdigit() or c in "+-." for c in cleaned)

    # Require at least 10 digits and valid symbols
    if digit_count >= 10 and valid_chars:
        phones_found.append(cleaned)
phones_found
```

Out[7]:
```
['555-123-4567', '+1-555-999-8888', '555.333.4444']
```

- We got there, but had to re-write most of the code
- Let's find out how we can solve these problems in a more compact and flexible way
- Using regular expressions, each of these problems can be solved with one line of code

In [8]:
```
import re

emails_regex = re.findall(r"[\w.-]+@[\w.-]+\.[a-zA-Z]{2,}", text)
emails_regex
```

Out[8]:
```
['john.doe@techcorp.com',
 's.connor@cyberdyne.io',
 'alex_99@dev-labs.org',
 'admin@hospital.net']
```

In [9]:
```
phones_regex = re.findall(r"\+?\d[\d.-]{8,}\d", text)
phones_regex
```

Out[9]:
```
['555-123-4567', '+1-555-999-8888', '555.333.4444']
```

What are regular expressions?

- From chapter 31 in Advanced Guide to Python 3 Programming
- Often just called regex
  - Strickly speaking regular expression is the concept in theoretical computer science
  - Regex is the practical implementation in software
- Introduction to using the `re` package in Python for regular expressions
- regex101 allows you to test out regular expressions
- If you open a text file in VS Code and press `Ctrl` + `F` / `Cmnd` + `F` and toggle the Regex option, you can highlight which parts of the text matches the specified pattern.

> Regular expression is a very powerful way of processing text while looking for recurring patterns

The basics for forming a regular expression

- Define a pattern
- Search for that pattern in a string
- Optionally, match groups within a string

- Optionally, create new string by replacing the pattern

Some simple Python examples

- First, we will use `re.search()` to look for a word in a string
- The first argument is the pattern we are looking for, and the second argument is the string

In [10]:
```
re.search("short", "This is a short line of text")
```

Out[10]:
```
<re.Match object; span=(10, 15), match='short'>
```

- This returned a re.Match object
- This will among other things give us the locations of the match in the string

In [11]:
```
m = re.search("short", "This is a short line of text")
print("String:", m.string)
print("Pattern:", m.re)
print("Match:", m.group())
print("Span:", m.span())
print("Start:", m.start())
print("Stop:", m.end())
```

```
String: This is a short line of text
Pattern: re.compile('short')
Match: short
Span: (10, 15)
Start: 10
Stop: 15
```

In [12]:
```
m = re.search("long", "This is a short line of text")
print(m)
```

```
None
```

In [13]:
```
re.search("is", "This is a short line of text")
```

Out[13]:
```
<re.Match object; span=(2, 4), match='is'>
```

- Note than when we searched for "is", we got the first instance of these two letters together,
  in other words the second half of the word "This"
- `re.search()` returns the first match
- We can find all matches by using `re.findall()`

In [14]:
```
re.findall("is", "This is a short line of text")
```

Out[14]:
```
['is', 'is']
```

In [15]:
```
re.sub("short", "long", "This is a short line of text")
```

Out[15]:
```
'This is a long line of text'
```

More advanced use

- By creating more flexible patterns, this becomes much more interesting!
- The `re` packages has several useful functions, for example:
  - `re.search(pattern, string)` finds first match of pattern in string
  - `re.match(pattern, string)` as `search()`, but match *only* at beginning of string
  - `re.findall(pattern, string)` finds all matches of pattern in string
  - `re.finditer(pattern, string)` iterate over all matches in string, returning match objects
  - `re.split(pattern, string)` split string on pattern and return list of parts
  - `re.sub(pattern, replacement, string)` replaces all occurances of pattern in string with replacement
  - `re.compile(pattern)` returns regular expression object, improves efficiency if same regex is used many times
- So far we have used regular strings as patterns, but typically you would want to use *raw* string
  - Regexes often contain backslashes, and using raw strings makes sure this is interpreted correctly by Python

In [16]:
```
r"This is a raw string. \n We use it to deal with \ in strings."
```

Out[16]:
```
'This is a raw string. \\n We use it to deal with \\ in strings.'
```

In [17]:
```
normal = "This is a string \n with a newline character inside."
print(normal)
```

```
This is a string
 with a newline character inside.
```

In [18]:
```
normal_extra = "This is a string \\n with a newline character inside."
print(normal_extra)
```

```
This is a string \n with a newline character inside.
```

In [19]:
```
raw = r"This is a string \n with a newline character inside."
print(raw)
```

```
This is a string \n with a newline character inside.
```

- Regexes are more powerful when we use flexible patterns
- Let's look for all words ending with the letter `e`

In [21]:
```
re.findall(r"\w*e\b", "I can see the e red rose and the purple lilac.")
```

Out[21]:
```
['see', 'the', 'e', 'rose', 'the', 'purple']
```

- Break-down of the pattern:
  - `\w`: Matches any character that can be part of a word (letters, numbers, underscore)

- `*` : Match this 0 or more times
- `e` : The lower-case letter e
- `\b` : Matches any word-boundary character

- Now, let's find all the words that are at least four letters long

In [25]:
```
re.findall(r"\b\w{4,4}\b", "I can see the red rose and the purple lilac.")
```

Out[25]:
```
['rose']
```

- `{4,}` means: previous symbol repeated at least four times
- More generally: `{m,n}` means at least `m` repetitions and at most `n` repetitions
- Now we look for three-word sequences beginning with "the"

In [26]:
```
re.findall(r"the\s+\w+\s+\w+\b", "I can see the red rose and the purple lilac.")
```

Out[26]:
```
['the red rose', 'the purple lilac']
```

- Break-down of the pattern:
  - `the` : The word "the" (all lower-case)
  - `\s+` : At least one occurance (`+`) of a whitespace (`\s`)
  - `\w+` : At least one occurance (`+`) of any word-characters
  - `\b` : A word-boundary characters

Grouping in regular expressions

- Use `()` to extract sub-components of a string
- Let's extract colour and plant names from the same string as before

In [27]:
```
re.findall(r"the\s+(\w+)\s+(\w+)\b", "I can see the red rose and the purple lila
```

Out[27]:
```
[('red', 'rose'), ('purple', 'lilac')]
```

- For each match of the entire pattern (e.g. "the red rose") we now get returned the parts in parentheses
- We can access the parts in the groups in the replacement strings
  - `\1` is the first group, `\2` the second

In [28]:
```
re.sub(r"the\s+(\w+)\s+(\w+)\b", r"a \2 [\1]", "I can see the red rose and the p
```

Out[28]:
```
'I can see a rose [red] and a lilac [purple].'
```

- We can also name the groups
  - Syntax: `(?P<colour>\w+)` will give the name `colour` to the group captured with `(\w+)`

In [29]:
```
re.sub(r"the\s+(?P<colour>\w+)\s+(?P<plant>\w+)\b",
       r"a \g<plant> [\g<colour>]",
       "I can see the red rose and the purple lilac.")
```

Out[29]:
```
'I can see a rose [red] and a lilac [purple].'
```

- Using flexible patterns and grouping makes the output of functions like `search()` much more interesting

In [30]:
```
m = re.search(r"the\s+(\w+)\s+(\w+)\b",
              "I can see the red rose and the purple lilac.")
print("Match object:", m)
print("Match string:", m.group())
print("Groups      :", m.groups())
```

```
Match object: <re.Match object; span=(10, 22), match='the red rose'>
Match string: the red rose
Groups      : ('red', 'rose')
```

In [31]:
```
m = re.search(r"the\s+(?P<color>\w+)\s+(?P<plant>\w+)\b",
              "I can see the red rose and the purple lilac.")
print("Match object:", m)
print("Match string:", m.group())
print("Groups      :", m.groups())
print("Group dict  :", m.groupdict())
```

```
Match object: <re.Match object; span=(10, 22), match='the red rose'>
Match string: the red rose
Groups      : ('red', 'rose')
Group dict  : {'color': 'red', 'plant': 'rose'}
```

- Let's try this with a text-file
- By using named groups, we get a dictionary of groups

In [32]:
```
pattern = r"(?P<Name>.+);(?P<Age>\d+);(?P<Phone>.+)"
# pattern = re.compile(r"(?P<Name>.+);(?P<Age>\d+);(?P<Phone>.+)")
with open("students_with_header.txt") as f:
    f.readline()
    students = [re.search(pattern, line).groupdict() for line in f]
students
```

Out[32]:
```
[{'Name': 'Per', 'Age': '24', 'Phone': '12345678'},
 {'Name': 'Kari', 'Age': '23', 'Phone': '23456789'}]
```

In [33]:
```
re.compile(r"(?P<Name>.+);(?P<Age>\d+);(?P<Phone>.+)")
```

Out[33]:
```
re.compile(r'(?P<Name>.+);(?P<Age>\d+);(?P<Phone>.+)', re.UNICODE)
```

- Now a more advanced example:
  - We can refer to a group inside a search pattern
  - Use this to look for repetitions of words (which probably are the result of a typo)

In [34]:
```
re.search(r"\b(\w+)\s+\1\b", "I can see the the red rose rose and the purple lil
```

Out[34]:
```
<re.Match object; span=(10, 17), match='the the'>
```

- Break-down of this pattern:
  - `\b` : a word boundary (we must start at the beginning of a word)
  - `(\w+)` : matches a non-empty word and collects it as into a group
  - `\s+` : at least one whitespace (if there is a comma or similar between words, repeated words are probably ok)
  - `\1` : that the word collected in the group must occur here again
  - `\b` : a word boundary
- Use information from the match object to mark location of problem.

In [35]:
```
text = "I can see the the red rose rose and the purple lilac."
m = re.search(r"\b(\w+)\s+\1\b", text)
print(text)
print('-' * m.start() + '^' * (m.end() - m.start()))
```

```
I can see the the red rose rose and the purple lilac.
----------^^^^^^^
```

- We can use `finditer()` to get one match at a time

In [36]:
```
for m in re.finditer(r"\b(\w+)\s+\1\b",
                     "I can see the the red rose rose and the purple lilac."):
    print(m)
```

```
<re.Match object; span=(10, 17), match='the the'>
<re.Match object; span=(22, 31), match='rose rose'>
```

In [37]:
```
text = "I can see the the red rose rose and the purple lilac."
print(text)
for m in re.finditer(r"\b(\w+)\s+\1\b", text):
    print('-' * m.start() + '^' * (m.end() - m.start()))
```

```
I can see the the red rose rose and the purple lilac.
----------^^^^^^^
---------------------^^^^^^^^^
```

- Note: Using groups in `re.findall()` changes the output type from a list of full matches to a list of captures group tuples

In [38]:
```
re.findall(r"the\s+(?P<color>\w+)\s+(?P<plant>\w+)\b", "I can see the red rose a
```

Out[38]:
```
[('red', 'rose'), ('purple', 'lilac')]
```

In [39]:
```
re.findall(r"the\s+\w+\s+\w+\b", "I can see the red rose and the purple lilac.")
```

Out[39]:
```
['the red rose', 'the purple lilac']
```

Greedy vs lazy matching

- Greedy matching
  - The default behavior in regular expressions

    - Matches as much text as possible before letting the rest of the pattern attempt to match
- Lazy (non-greedy) matching
    - Flips this behavior
    - Matches as little text as possible, expanding one character at a time only when the rest of the pattern fails.
- Let's look at a couple of examples
    - `*` - Match preceding character any number of times
    - `?` - Match preceding character zero or one time
    - `.` - Matches any character
- Appending `?` to any standard quantifier converts it from greedy to lazy

In [40]:
```
html_str = "<b>hello</b> and <b>world</b>"
html_str
```

Out[40]:
```
'<b>hello</b> and <b>world</b>'
```

In [ ]:
```
# Greedy: grabs from the first <b> to the LAST </b>
greedy_match = re.findall(r"<b>.*</b>", html_str)
print("Greedy:", greedy_match)
```

```
Greedy: ['<b>hello</b> and <b>world</b>']
```

In [42]:
```
# Lazy: grabs from first <b> to the NEXT </b>
lazy_match = re.findall(r"<b>.*?</b>", html_str)
print("Lazy:  ", lazy_match)
```

```
Lazy:   ['<b>hello</b>', '<b>world</b>']
```

Description	Greedy	Lazy	Behavior
0 or more	`*`	`*?`	Greedy eats all; Lazy takes 0 first
1 or more	`+`	`+?`	Greedy eats all; Lazy takes 1 first
0 or 1	`?`	`??`	Greedy takes 1; Lazy takes 0
Range	`{n,m}`	`{n,m}?`	Greedy tries m; Lazy tries n

Flags

- By adding flags to the functions you can change the way the matches work
- `re.ASCII` or `re.A`: Makes several escapes like \w, \b, \s and \d match only on ASCII characters.
- `re.DOTALL`, `re.S`: Make `.` match any character, including newlines.
- `re.IGNORECASE`, `re.I`: Do case-insensitive matches.
- `re.MULTILINE`, `re.M`: Multi-line matching, affecting `^` and `$`.
    - `^` indicates the start of a line
    - `$` indicated the end of a line
- `re.VERBOSE`, `re.X`: Enable verbose REs, which can be organized more cleanly and understandably.

In [43]:
```
re.findall(r"the", "The beauty and the beast", flags=re.IGNORECASE)
```

Out[43]:
```
['The', 'the']
```

In [44]:
```
date_text = "2026-10-10"

# Without re.VERBOSE (Hard to read and maintain)
dense_pattern = r"^(19|20)\d{2}-(0[1-9]|1[0-2])-(0[1-9]|[12]\d|3[01])$"

# With re.VERBOSE (Self-documenting and organized)
verbose_pattern = re.compile(
    r"""
    ^                               # Start of string
    (?P<year> (?:19|20)\d{2} )      # 4-digit year (1900-2099)
    -                               # Literal dash separator
    (?P<month> 0[1-9]|1[0-2] )      # 2-digit month (01-12)
    -                               # Literal dash separator
    (?P<day> 0[1-9]|[12]\d|3[01] )  # 2-digit day (01-31)
    $                               # End of string
""",
    re.VERBOSE,
)

match = verbose_pattern.search(date_text)
print(match.groupdict())
```

```
{'year': '2026', 'month': '10', 'day': '10'}
```

- `re.VERBOSE` is useful when you write complex patterns

An overview of the most used symbols in patterns

Symbol	Description
`.`	Matches any single character except newline
`*`	Match preceding character any number of times
`+`	Match preceding character one or more times
`?`	Match preceding character zero or one time
`{n}`	Match preceding character exactly n times
`{n, m}`	Match preceding character at least n, at most m times
`[abc]`	Match any of the characters in the brackets

| `[^abc]` | Match any character not given in the brackets | `[a-z]` | Match any character from a-z | | `\b` | Match word boundary | | `\d` | Match digit | | `\s` | Match whitespace | | `\w` | Match word character (`[A-Za-z0-9_]` and all characters defined as letters by Unicode) | | `^` or `\A` | Matches beginning of input | | `$` or `\Z` | Matches end of input | | `()` | Group, can be referenced later | | `(?P<name>)` | Named group | | `\1` , `\2` , ... | Reference to group defined earlier | | `\(` , `\)` | Required to include parentheses in pattern |

Now we can re-visit the motivational example we started with:

Email Regex Pattern Breakdown

Pattern: `r"[\w.-]+@[\w.-]+\.[a-zA-Z]{2,}"`

```
  [\w.-]+  @  [\w.-]+  \.  [a-zA-Z]{2,}
  |        |  |        |   |
  |        |  |        |   └── Top-Level Domain (TLD): 2+ letters
(e.g., com, org, io)
  |        |  |        └────── Literal Dot: Escaped (\.) because .
means "any character"
  |        |  └─────────────── Domain Name: 1+ word characters, dots,
or hyphens
  |        └────────────────── Literal '@' symbol
  └─────────────────────────── Username / Local Part: 1+ word
characters, dots, or hyphens
```

- `[\w.-]+` (Username / local part)
  - `[...]` — Character class matching any single character inside the brackets.
  - `\w` — Shorthand for word characters (a-z, A-Z, 0-9, _).
  - `.` and `-` — Matches a literal dot or hyphen (inside character classes [...], the dot automatically loses its special "match everything" power).
  - `+` — Quantifier for 1 or more characters.
- `@` (Literal Separator)
  - Matches the single `@` character strictly once.
- `[\w.-]+` (domain name)
  - One or more word characters, dots, or hyphens (handles subdomains like dev-labs or mail.techcorp).
- `\.` (domain separator)
  - Escaped backslash `\.` forces a literal dot. Outside of square brackets, an unescaped dot `.` matches any character (which would accidentally match user@domainXcom).
- `[a-zA-Z]{2,}` (top-level domain / TLD)
  - `[a-zA-Z]` — Restricts TLDs to uppercase and lowercase letters only (excluding digits or underscores).
  - `{2,}` — Quantifier requiring at least 2 characters (e.g., .io, .com, .museum).
- Note: This pattern does not capture all possible email addresses, the full pattern for that would be very long and complex.

Phone Number Regex Pattern Breakdown

Pattern: `r"\+?\d[\d.-]{8,}\d"`

```
  \+?   \d   [\d.-]{8,}   \d
  |     |    |            |
```

```
  │     │    │            └── Trailing Digit: Ensures sequence ends on
a digit (ignores '!' or '.')
  │     │    └─────────────── Middle Sequence: 8+ characters of digits,
dots, or hyphens
  │     └──────────────────── Leading Digit: Guarantees sequence starts
with a number
  └────────────────────────── Optional Plus Sign: Matches '+' for
international codes (0 or 1 time)
```

- `\+?` (Optional Country Code Indicator)
  - `\+` — Escaped plus sign (since `+` is normally a quantifier meaning "1 or more").
  - `?` — Quantifier meaning 0 or 1 time (allows numbers with or without a leading +).
- `\d` (Leading Digit)
  - Shorthand for any digit (0-9). Ensures the number starts with a digit immediately after the optional `+`.
- `[\d.-]{8,}` (Middle Digits & Separators)
  - `[...]` — Character class matching digits (`\d`), literal dots (`.`), or hyphens (`-`).
  - `{8,}` — Quantifier requiring at least 8 of these characters in sequence.
- `\d` (Trailing Digit)
  - Requires the final character to be a digit (0-9). This prevents trailing sentence punctuation (like call 555-123-4567!) from being incorrectly captured inside the phone match.

When not to regex?

- Sometimes your problem will not need a full regex solution
- `str.split()` will split a string into words
- Use `str.startswith()` or `str.endsswith()` to check the start or end of a string